

1 solving congruences

imagine it is 10 o'clock. 5 hours later, it is 3 o'clock, not 15. this "resetting" behaviour is the essence of **modular arithmetic**.

1.1 arithmetic operations with $\text{mod } n$

$\mathbb{Z}_m$ denotes the set of non-negative integers less than m (the remainders when dividing by m).

modular arithmetic "preserves" addition and multiplication:

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$$

we define operations restricted to this set:

$$a +_m b = (a + b) \bmod m$$

$$a \cdot_m b = (a \cdot b) \bmod m$$

1.2 solving linear congruences

we look at equations of the form $ax \equiv b \pmod{m}$.

1.2.1 additive inverses

look at an example: in order to solve $a + 8 \equiv 2 \pmod{11}$:

1. find the **additive inverse** of 8 modulo 11.

- $-8 \equiv 3 \pmod{11}$
- $-8 = (11 \cdot -1) + 3$

2. add 3 to both sides: $a \equiv 5 \pmod{11}$.

note: adding any multiple of m preserves the value $\pmod{m}$.

1.2.2 multiplicative inverses

to solve $ax \equiv b \pmod{m}$ (a **linear congruence**), given values for a and b , we need a value $\bar{a}$ such that $a \cdot \bar{a} \equiv 1 \pmod{m}$. if we have this value, we can multiply it on both sides (similarly to multiplying by the reciprocal), and then simplify to find the solution. bézout's theorem, introduced below, helps to find that number.

theorem (bézout's theorem): if $\gcd(a, m) = 1$ (a and m are coprime), there exist integers s and t such that: $sa + tm = 1$.

this implies $sa \equiv 1 \pmod{m}$, meaning s is the multiplicative inverse of a .

collary: since $\gcd(a, m) = 1$ and $sa + tm = 1$, by substitution, we can get the **bézout identity**: $\gcd(a, b) = sa + tb$.

1.3 extended euclidean algorithm (eea)

the extended euclidean algorithm builds on the basic version that we have learned in the previous lecture. other than just finding the gcd, we also find the **bézout** numbers (s, t ; refer to section above).

the first step of the extended euclidean algorithm is just by performing the basic version first; let's use $a = 99$ and $b = 78$ as an example.

$$\begin{aligned} 99 &= 78 \cdot 1 + 21 \\ 78 &= 21 \cdot 3 + 15 \\ 21 &= 15 \cdot 1 + 6 \\ 15 &= 6 \cdot 2 + 3 \\ 6 &= 2 \cdot 3 + 0 \end{aligned}$$

since the last remainder is zero(0), we disregard the last line and use the line before: $\gcd(99, 78) = 3$.

to find out the **bézout** numbers for the second step of the algorithm, we want to find how to express 3 with 99 and 78. to do that, we look at the previous work, start from the bottom, and work our way back.

$$\begin{aligned} 3 &= 15 - (2 \cdot 6) \\ 6 &= 21 - (1 \cdot 15) \\ 3 &= 15 - (2 \cdot (21 - (1 \cdot 15))) && \text{(substitute 6)} \\ &= 15 - (2 \cdot 21) + (2 \cdot 15) && \text{(distribute -2)} \\ &= (3 \cdot 15) - (2 \cdot 21) && \text{(combine 15s, rearrange)} \\ 15 &= 78 - (3 \cdot 21) \\ 3 &= (3 \cdot (78 - (3 \cdot 21))) - (2 \cdot 21) && \text{(substitute 15)} \\ &= (3 \cdot 78) - (9 \cdot 21) - (2 \cdot 21) && \text{(distribute 3)} \\ &= (3 \cdot 78) - (11 \cdot 21) && \text{(combine 21s)} \\ 21 &= 99 - (1 \cdot 78) \\ 3 &= (3 \cdot 78) - (11 \cdot (99 - (1 \cdot 78))) && \text{(substitute 21)} \\ &= (3 \cdot 78) - (11 \cdot 99) + (11 \cdot 78) && \text{(distribute -11)} \\ 3 &= (\boxed{14} \cdot 78) + (\boxed{-11} \cdot 99) && \text{(combine 78s)} \end{aligned}$$

we have found that $s = -11$ and $t = 14$ such that $3 = sa + tb$.

here's an iterative version of the algorithm in pseudocode:

algorithm: extended euclidean algorithm (eea)

```

1 procedure extended_euclid(a, b)
2   (oldr, r) := (a, b)
3   (olds, s) := (1, 0)
4   (oldt, t) := (0, 1)
5   while r ≠ 0
6     q := ⌊oldr/r⌋
7     (oldr, r) := (r, oldr - q * r)
8     (olds, s) := (s, olds - q * s)
9     (oldt, t) := (t, oldt - q * t)
10  return (oldr, olds, oldt)

```

1.4 chinese remainder theorem (crt)

used to solve systems of congruences where moduli are pairwise coprime.

assume we have a system like this:

$$\begin{aligned}
 x &\equiv a_1 \pmod{m_1} \\
 x &\equiv a_2 \pmod{m_2} \\
 &\dots \\
 x &\equiv a_n \pmod{m_n}
 \end{aligned}$$

solution: let $m = m_1 m_2 \dots m_n$. let $M_k = \frac{m}{m_k}$ (product of all moduli except m_k). let y_k be the inverse of M_k modulo m_k .

the unique solution modulo m is:

$$x = \sum_{k=1}^n a_k M_k y_k$$

example (sunzi suanjing):

$$x \equiv 2 \pmod{3}, x \equiv 3 \pmod{5}, x \equiv 2 \pmod{7}$$

- $m = 3 \cdot 5 \cdot 7 = 105$.
- $M_1 = 35, y_1 = 2$ (since $35 \cdot 2 = 70 \equiv 1 \pmod{3}$).
- $M_2 = 21, y_2 = 1$ (since $21 \cdot 1 = 21 \equiv 1 \pmod{5}$).
- $M_3 = 15, y_3 = 1$ (since $15 \cdot 1 = 15 \equiv 1 \pmod{7}$).
- $x = 2(35)(2) + 3(21)(1) + 2(15)(1) = 140 + 63 + 30 = 233 \equiv 23 \pmod{105}$.

1.5 advanced concepts

1.5.1 fermat's little theorem

if p is prime and p does not divide a :

$$a^{p-1} \equiv 1 \pmod{p}$$

useful for computing large powers, e.g., $7^{222} \bmod 11$.

1.5.2 primitive roots & discrete log

- **primitive root (r):** an element in $\mathbb{Z}_p$ whose powers generate all nonzero elements of $\mathbb{Z}_p$.
- **discrete log problem:** given p, r, b , find e such that $r^e \equiv b \pmod{p}$. there is no known polynomial time algorithm for this (foundation of cryptography).